

A Copy-and-Patch JIT for Threaded Interpreters

Nishil Kapadia^{1,†}, Dave Mason^{1,*,†}

¹Toronto Metropolitan University, Toronto, Canada

Abstract

Threaded interpreters are a practical foundation for dynamic language virtual machines, but their frequent indirect dispatch incurs significant execution overhead. At the other end of the design spectrum, optimizing JIT compilers can reduce this cost but require substantial compiler infrastructure and long-term engineering investment. This paper presents a copy-and-patch JIT compiler for threaded interpreters in the Zag Smalltalk VM, implemented in Zig, as a middle-tier alternative between pure interpretation and full optimizing compilation. Our approach reuses existing threaded functions as machine-code templates. At runtime, the system extracts patchable threaded-function regions, copies them into executable memory, relocates PC-relative instructions, and rewrites control-flow edges to construct method-specific native execution paths. The current version targets a bounded but practical subset of Smalltalk threaded functions, including stack operations, control-flow operations, inline arithmetic primitives, and send/return paths. On AArch64, the copy-and-patch tier preserves observed behavior and delivers consistent speedups of about five percent on tested Fibonacci inputs, while dispatch-heavy micro-benchmarks reach up to 1.54× speedup over the threaded baseline. These gains remain bounded by send-heavy execution paths and by the current prototype implementation.

Keywords

Threaded Interpreters, JIT Compiler, Dynamic Language Runtime, Code Generation,

1. Introduction

Threaded execution/interpretation is a compelling implementation strategy for dynamic language runtimes because it is compact, portable, and easy to maintain [1, 2]. Their persistent cost is dispatch overhead: execution repeatedly transfers control indirectly among threaded functions, stressing branch prediction and instruction delivery [2, 3]. At the opposite end of the design space, optimizing JIT compilers can remove much of this overhead, but at substantial engineering cost, including IR design, analysis pipelines, register allocation, deoptimization machinery, and long-term maintenance across architectures and language features. In practice, this leaves a gap between “fast enough to prototype” and “fast enough to deploy.”

This work explores a middle tier in that gap: a copy-and-patch JIT for threaded execution [4]. Rather than introducing a new IR and backend, we reuse existing threaded functions as native-code templates. The original work created templates off-line equivalent to interpreter byte-codes. They then built up executable methods/functions by concatenating the templates corresponding to the byte-codes in the original method.

Since we already have threaded functions executed in sequence to implement the method, we have built-in templates available. At runtime, our system reuses those existing threaded functions as templates for method-specific code generation. For each threaded code in the compiled method, we resolve the corresponding threaded function entry, analyze that function to determine a copy size and patchable branch offsets, and then copy the selected threaded-function bytes into executable memory. During this copy phase, PC-relative instructions are relocated to the new addresses, and branch sites are rewritten so control flow follows the method’s branch targets. This produces a contiguous native path assembled from existing threaded code. Conceptually, the approach preserves interpreter semantics while replacing repeated indirect dispatch with direct control flow.

IWST-2026: International Workshop on Smalltalk Technology, 7-10 July 2026, Plovdiv, Bulgaria

*Corresponding author.

†These authors contributed equally.

✉ n1kapadia@torontomu.ca (N. Kapadia); dmason@torontomu.ca (D. Mason)

🆔 0000-0002-2688-7856 (D. Mason)



© 2026 Copyright for this paper by its authors. Use permitted under Creative Commons License Attribution 4.0 International (CC BY 4.0).

The key technical challenge is deriving reliable templates from compiled threaded functions without fragile heuristics. We address this with control-flow-guided template scanning. Starting from a function entry, we traverse reachable instructions, identify terminators and branch forms, and recover patchable branch offsets. This yields template boundaries and patch locations that remain robust under routine code-layout variation. During template copy phase, we relocate architecture-dependent addressing modes and rewrite selected indirect branches into direct branches targeting copied successors. Because the process operates on existing machine code, most implementation complexity is concentrated in a compact architecture-specific relocation and patch layer rather than a full compiler stack.

In Zag, this makes copy-and-patch a practical alternative to heavier JIT paths such as LLVM-based compilation [5], while remaining within the broader Smalltalk implementation lineage that spans classic systems and richer JIT designs [6, 7]. The system lets us evaluate whether copy-and-patch integrates cleanly with an existing threaded Smalltalk runtime and whether reducing indirect dispatch yields measurable gains on both method-level workloads and microbenchmarks.

2. Threaded Interpretation

Zag is a Smalltalk VM whose execution core is a threaded interpreter. A compiled method is represented as a linear code stream over opcode handlers plus operands (literals, variables, signatures, branch labels). Execution advances by transferring control among native threaded functions, each of which operates on a common VM state tuple: program counter (PC), stack pointer (SP), process, context, and method-local metadata (Extra). This organization matches the general structure of efficient threaded interpreters described in prior work [1, 2, 8].

At the implementation level, Zag’s threaded functions share a uniform ABI:

```
pub const ThreadedFn = *const fn (PC, SP, *Process, *Context, Extra) Result;
```

This single function shape is important: it allows opcode handlers such as `pushLiteral`, `branchFalse`, `send`, and `returnTop` to compose directly at runtime without introducing an intermediate IR executor.

2.1. Threaded Functions

At runtime, behavior is realized by concrete threaded functions. The following implementation-level excerpt shows Zag’s `branchFalse` path in terms of stack inspection, PC target resolution, branch checking, and tail-call dispatch:

```
pub fn threadedFn(pc: PC, sp: SP, process: *Process, context: *Context, extra:
↪ Extra) Result {
    if (sp.top.equals(False())) {
        const newPc = pc.targetPC();
        return @call(tailCall, process.branchCheck(newPc.prim()),
            .{newPc.next(), sp.drop(), process, context, extra});
    }

    return @call(tailCall, process.check(pc.prim2()), .{
        pc.next2(), sp.drop(), process, context, extra,
    });
}
```

This implementation is specific to Zag’s threaded runtime model. It evaluates the top-of-stack boolean predicate, computes the taken target via `targetPC()`, and then tail-calls the next handler `.`. In both paths, the operand is consumed (`sp.drop()`) and control resumes at the opcode-specific successor PC.

2.2. Dispatch

Dynamic dispatch remains the semantic center of execution. In Zag, method invocation proceeds through the executor’s `sendTo` path, which resolves and executes methods under standard Smalltalk `send` semantics:

```

_ = @call(.never_inline, MainExecutor.sendTo, .{
    exe,
    Sym.fibonacci.asObject(),
    exe.object(@as(i56, fib_n)),
}) catch unreachable;

```

Dynamic dispatch in Zag is implemented inside the threaded send path rather than at a separate dispatch layer. When a send opcode is reached, Zag reads the call-site signature from the bytecode stream, inspects the receiver’s class, resolves the corresponding compiled method, records the return continuation, and then tail-calls the resolved method entry.

```

inline fn getMethod(pc: PC, signature: Signature, receiver: Object)
    *const CompiledMethod {
    const class = receiver.get_class();
    const requiredSignature = signature.withClass(class);
    if (signature == requiredSignature) return pc.next().method();
    return class.lookupMethodForClass(requiredSignature);
}

```

Zag’s interpreter organization is a strong fit for copy-and-patch JIT compilation: threaded functions already exist as native code fragments with a stable ABI and well-defined control-flow boundaries. As a result, our JIT tier does not replace Zag’s Smalltalk semantics or method model. Instead, it copies existing threaded functions into method-specific native paths by copying templates and patching control-flow edges. The motivation for targeting these indirect transfers is consistent with prior work showing that efficient interpreters are unusually sensitive to indirect branch behavior [2, 3].

3. Copy-and-Patch

Our primary goal is to reduce threaded-dispatch overhead in Zag without introducing the complexity of a full optimizing compiler. The design is intentionally pragmatic: reuse existing threaded functions, preserve current VM semantics, and keep implementation surface area small. This positions the tier close in spirit to copy-and-patch compilation [4], but adapted to an existing Smalltalk VM rather than a separately constructed stencil library.

Zag already executes Smalltalk methods through threaded functions with a stable runtime ABI (PC, SP, Process, Context, Extra). The copy-and-patch tier sits directly above this baseline interpreter path. It does not alter object representation, method lookup, or message-send semantics. Instead, it changes how control moves between threaded functions for a given method instance.

Conceptually:

- Threaded baseline: repeated indirect transitions among threaded functions.
- Copy-and-patch: method-specific native path assembled from the same threaded functions with patched direct edges.

3.1. Pipeline

For each compiled method, the tier runs four steps:

1. Analyze the method: read opcode order and branch targets from Zag’s tuple form.
2. Prepare templates: collect threaded-function templates and patch-site metadata.
3. Build JIT code: copy templates into a contiguous executable buffer and patch control flow.
4. Install entries: rewrite method opcode entries to point to JIT entry points.

Figure 1 summarizes this workflow.

- Threaded-function logic is reused, not reimplemented.

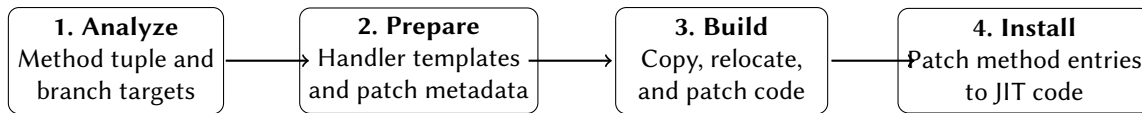


Figure 1: High-level copy-and-patch pipeline for a compiled method in Zag.

- Runtime state and calling convention are unchanged.
- Message send/return behavior remains under existing Zag mechanisms.
- Only control-flow linkage between reused threaded functions is specialized.

The current prototype targets a practical subset of Zag Smalltalk bytecodes: stack operations, control-flow bytecodes, inline arithmetic primitives, and send/return paths. This subset captures representative interpreter behavior. Subsequent sections detail the mechanisms about this design.

4. Template Extraction

Copy-and-patch in Zag depends on recovering safe, patchable machine-code templates from existing threaded functions. This section describes how template boundaries and branch patch sites are extracted on AArch64 without relying on fragile heuristics.

Each handler is summarized as:

```

pub const TemplateInfo = struct {
    start: [*]const u8,
    size: usize,
    br_offsets: [8]usize,
    br_count: usize,
    last_branch_offset: ?usize,
};
  
```

`size` gives the copied region length; `br_offsets` records candidate patch sites; `last_branch_offset` is used for linear fallback chaining.

For a given handler function pointer, extraction first finds the next known threaded-function symbol and uses that address as a preferred upper bound.

If unavailable, the scanner uses a conservative hard cap (32 KiB). This prevents unbounded disassembly while remaining robust to function layout variation.

4.1. Control-Flow Graph Scan

The core scanner performs a control-flow traversal from offset 0 over machine instructions.

It tracks:

- discovered basic-block entries (queue)
- visited blocks/instructions (`block_seen`, `inst_seen`)
- maximum reachable offset (`out.size`)
- branch-register patch sites (`out.br_offsets`)

Unlike linear "scan until return" approaches, traversal follows explicit control transfers and therefore handles internal branches inside handlers.

Instruction Classes The AArch64 scanner classifies instructions into control-flow categories:

- `BR Xn` (indirect branch register): recorded as patchable dispatch site.
- `RET`: terminates block.
- `B imm`: enqueue branch target, terminate current block.

Original threaded functions

```
0x5dc4 pushLiteral
...
0x5e08 br x5          ; indirect transfer

0x75c0 inline-I
...
0x7610 br x5          ; indirect transfer

0x8d50 branchFalse
...
0x8d90 br x5          ; indirect transfer

0xa210 returnTop
```

JIT buffer

```
0x0000 pushLiteral
...
0x0050 b 0x0064       ; patched direct branch

0x0064 inline-I
...
0x00b8 b 0x00d0       ; patched direct branch

0x00d0 branchFalse
...
0x0110 b 0x0140 / b 0x0120

0x0140 returnTop
```

Figure 2: In threaded execution, continuation moves indirectly among threaded functions at scattered addresses. In the copy-and-patch tier, the same method fragment is laid out in one contiguous JIT buffer, and continuation becomes a sequence of patched direct branches.

- B.cond, CBZ/CBNZ, TBZ/TBNZ: enqueue target and fallthrough successor.

These instruction classes follow standard AArch64 control-flow encodings as defined by Arm’s architectural specification [9].

This logic is implemented with small encoding predicates/decoders (isBImm, isBCond, decode*Imm, etc.) and keeps extraction architecture-local while VM logic remains unchanged.

Zag’s threaded functions naturally end in control transfers to the next interpreter action. By scanning reachable CFG regions and recording branch-register sites, extraction recovers the information needed by the patcher. (1) copy extent for each threaded-function template. (2) concrete offsets where direct edges can replace indirect dispatch.

5. Relocation and Branch Patching on AArch64

After template extraction, the copy-and-patch tier creates method-specific native code by copying Zag threaded-function templates into a JIT buffer and rewriting selected control-flow edges. This section focuses on the AArch64 mechanisms used to make copied templates executable and correctly linked.

Figure 2 makes the layout effect explicit. In the threaded baseline, continuation moves among threaded functions at unrelated machine addresses. In the JIT path, the same logical steps are copied into one contiguous region, and continuation is expressed with local direct branches. The important transformation is not generation of new instruction sequences, but redirection of existing control flow. The copied template body is preserved, while selected indirect transfers are rewritten into direct branches targeting neighboring templates in the JIT buffer.

5.1. Executable Memory

For each method instance, the runtime allocates an executable buffer, copies templates in opcode order, and records each copied template’s start offset. On macOS/AArch64, this uses JIT-capable mappings and explicit write/execute transitions.

```
var jit = try JitBuffer.init(total_size + 4096);
jit.makeWritable();

offsets[i] = jit.copyTemplate(info);
// ...
jit.patchBranch(curr_offset, br_offset, target_offset);

jit.makeExecutable();
```

In our implementation, JIT code generation allocates a buffer, makes it writable, copies the selected templates into it, and patches their control-flow edges. After that, the runtime invalidates the instruction cache and marks the buffer executable before running the generated code.

5.2. Relocation of PC-Relative Instructions

Because templates are moved from original code locations to the JIT buffer, PC-relative instructions must be adjusted. The current implementation relocates key AArch64 classes used in Zag threaded functions: ADRP (page-relative addressing) and B / BL (immediate branch/call).

The encoding and relocation behavior of these instructions follows the AArch64 definitions in the Arm Architecture Reference Manual [9].

Representative relocation logic:

```
fn relocateInst(orig_pc: usize, new_pc: usize, inst: u32) u32 {
    if (Arm64.isAdrp(inst)) { ... }
    else if (Arm64.isBImm(inst) or Arm64.isBlImm(inst)) { ... }
    return inst;
}
```

Relocation is applied instruction-by-instruction during template copy.

5.3. Branch Patching

Relocation preserves internal correctness of copied templates; branch patching links templates into a method-specific path. Patch sites come from template metadata produced by scanning. At each site, an indirect transfer is replaced with a direct AArch64 B imm to the chosen successor.

```
pub fn patchBranch(self: *Self, templateOffset: usize, brOffset: usize,
    ↪ targetOffset: usize) void {
    const patchAddr = templateOffset + brOffset;
    const offset: i32 = @intCast(@as(i64, @intCast(targetOffset)) - @as(i64,
    ↪ @intCast(patchAddr)));
    const inst = Arm64.encodeBranch(offset);
    std.mem.writeInt(u32, self.memory[patchAddr..][0..4], inst, .little);
}
```

This is the core dispatch-reduction step: runtime-selected direct edges replace repeated interpreter-style indirect transfer points.

Opcode-Specific Patching Target selection follows Zag opcode class semantics:

- Sequential ops (push, pushLiteral, dup, drop): patch to next copied template.
- Conditional branches (branchFalse, branchTrue): patch taken and fallthrough exits.
- Unconditional branch (branch): patch to label-resolved target.
- Inline primitives: patch success/fallback continuations according to fail-slot layout.
- Sends/returns/fail: preserve normal runtime behavior where direct chaining is not appropriate.

This policy is encoded in `JitMethod.init()` and applied after all template offsets are known.

5.4. JIT Installation

After patching, method opcode entries are rewritten to point at JIT buffer entry addresses:

```
for (info.positions, 0..) |pos, i| method.patchOp(compiled.code[0..], i, pos);
compiled.executeFn = method.getEntryFor(0);
```

Execution still uses Zag's existing runtime ABI and VM state transitions. The change is purely in realized control flow: a method-specific native path replaces much of the baseline indirect dispatch sequence.

6. Experimental Design

We will use one primary workload (Fibonacci) and a set of synthetic microbenchmarks to study how copy-and-patch behaves across different opcode and control-flow patterns.

The primary workload is Smalltalk Fibonacci, executed through Zag’s standard send semantics. We compare:

- Threaded: baseline Zag threaded interpreter path.
- CnP-JIT: same method with opcode entries patched to buffer template paths.

Both configurations share the same method definition, literals, and runtime interface and only the will control-flow will differ.

6.1. Fibonacci

To ground the experimental pipeline, we use Zag’s Smalltalk Fibonacci method as a running example.

```
fibonacci
  self <= 1 ifTrue: [ ^ self ].
  ^ (self - 1) fibonacci + (self - 2) fibonacci
```

In Zag’s threaded representation, this method is encoded as an opcode/operand tuple:

```
const tup = .{
  tf.push,                self,
  tf.pushLiteral,         "1const",
  tf.@"inline<=I",       tf.fail, tf.fail,
  tf.branchFalse,         "false",
  tf.returnSelf,          ":false",
  tf.push,                self,
  tf.pushLiteral,         "0const",
  tf.@"inline-I",         tf.fail, tf.fail,
  tf.send,                sig(.fibonacci, 0),
  &nullMethod,            tf.push,
  self,                   tf.pushLiteral,
  "1const",
  tf.@"inline-I",         tf.fail, tf.fail,
  tf.send,                sig(.fibonacci, 0),
  &nullMethod,
  tf.@"inline+I",         tf.fail, tf.fail,
  tf.returnTop,
};
```

This example exercises the key features targeted by the copy-and-patch tier: stack manipulation, conditional branch, inline arithmetic primitives, recursive sends, and return paths. We refer back to this method when interpreting template extraction, patching behavior, and evaluation outcomes.

For each n , we collect execution metrics and JIT construction metrics. Execution metrics include per-run latency for both threaded and JIT execution, along with summary statistics (median, mean, standard deviation). JIT construction metrics include templates copied, bytes copied, patched branches, and total build time.

6.2. Microbenchmarks

To complement Fibonacci, we evaluate a set of synthetic benchmarks in `cnp-bench`. These benchmarks isolate specific opcode and control-flow patterns so we can observe where copy-and-patch provides the largest benefit.

For each benchmark, we run 3 warmup rounds and 8 measured rounds, with 10,000 iterations per round. Reported values are median ns/iteration, with min–max range across measured rounds.

Table 1

Fibonacci runtime: threaded baseline (IntegerBr) vs copy-and-patch (IntegerCnP).

n	IntegerBr (ms)	IntegerCnP (ms)	Δ (ms)	Gain (%)	Speedup
32	57	54	3	5.3	1.056 $\times$
34	151	143	8	5.3	1.056 $\times$
36	394	374	20	5.1	1.053 $\times$
38	1034	986	48	4.6	1.049 $\times$
40	2703	2566	137	5.1	1.053 $\times$
42	7090	6706	384	5.4	1.057 $\times$

Benchmarks.

- **loop**: branch-heavy countdown benchmark with a backedge; stresses conditional and unconditional control-flow patching.
- **arith**: inline integer arithmetic benchmark; stresses primitive fast paths and continuation edges.
- **2**: minimal benchmark (`pushLiteral`, `returnTop`); lower-bound overhead case.
- **10**: short straight-line stack benchmark; dispatch sensitivity at small length.
- **20**: medium straight-line stack benchmark.
- **50**: longer straight-line stack benchmark.

These are mechanism-focused benchmarks rather than application-level workloads. Fibonacci remains our primary end-to-end workload, while `cnp-bench` provides controlled evidence for where copy-and-patch improves performance and where it does not.

7. Results**7.1. Fibonacci Performance**

Table 1 runtime for the threaded baseline (IntegerBr) and copy-and-patch execution (IntegerCnP) over $n \in \{32, 34, 36, 38, 40, 42\}$.

Copy-and-patch is faster than the threaded integer baseline at every tested input size. Across `fib(32)` through `fib(42)`, the improvement remains fairly stable at roughly five percent, while the absolute time saved grows substantially with problem size. This behavior is consistent with a local optimization that reduces a recurring portion of interpreter overhead in each send. As the recursion tree grows, the same method body is executed many more times, so even a modest per-send saving accumulates into a meaningful wall-clock reduction.

At the same time, Fibonacci is not a best-case workload for this technique. It remains heavily dominated by dynamic message sends, and those sends are still indirect under the current design. Copy-and-patch reduces the surrounding interpreter dispatch on stack, branch, primitive, and return paths, but it does not eliminate the cost of dynamic send resolution. The measured steady-state gains should therefore be interpreted as partial dispatch reduction rather than complete removal of dynamic dispatch overhead. This is an important limitation, but it also makes Fibonacci a conservative benchmark: the fact that copy-and-patch still improves performance in a send-heavy setting strengthens the case that the optimization is meaningful in realistic Smalltalk execution.

We also report runtime traffic counters for a `fib(40)` run. The benchmark records `sendCalls=204,668,308` and `getMethod=204,668,308`, indicating that dynamic sends dominate execution volume. Inline-cache behavior is stable in this run (`icHit=204,668,308`, `lookups=0`, `icPatch=0`), so the remaining cost is not cache-miss lookup traffic. Context pressure is also high (`ctxInstall=102,334,154`, `ctxReuse=102,334,154`). Most importantly, measured send dispatch time is `sendDispatch~3939.75ms`, accounting for `share~51.9%` of total runtime.

These counters support the main interpretation of our results that dynamic send dispatch consumes a large fraction of time on send-heavy workloads.

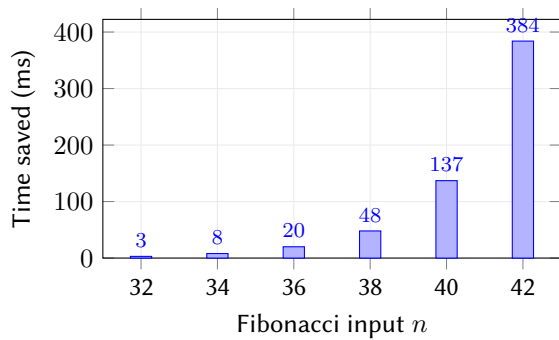
Table 2Dispatch traffic counters for a representative `fib(40)` run.

Metric	Value
sendCalls	204,668,308
getMethod	204,668,308
icHit	204,668,308
lookups	0
icPatch	0
ctxInstall	102,334,154
ctxReuse	102,334,154
sendDispatch time	3939.75 ms
sendDispatch share	51.9%

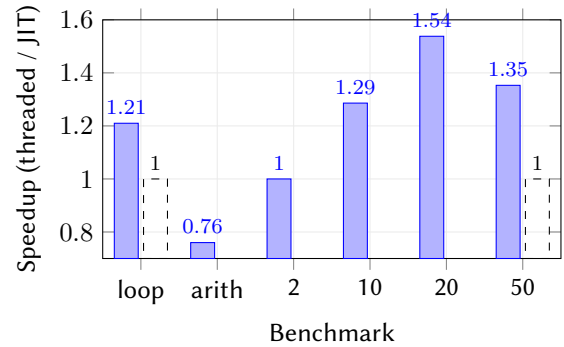
Table 3

CNP microbenchmarks (cnp-bench) in JIT vs threaded mode (ReleaseFast).

Case	Threaded (ns)	JIT (ns)	Δ (ns)	Gain (%)	Speedup
loop	342882	283327	59555	17.4	1.210 $\times$
arith	19	25	-6	-31.6	0.760 $\times$
2	2	2	0	0.0	1.000 $\times$
10	9	7	2	22.2	1.286 $\times$
20	20	13	7	35.0	1.538 $\times$
50	46	34	12	26.1	1.353 $\times$



(a) Fibonacci absolute time saved.



(b) Microbenchmark speedups.

Figure 3: (a) absolute time saved over the Fibonacci values (b) speedup distribution across microbenchmarks.

7.2. Microbenchmarks

Table 3 shows that copy-and-patch improves most control-flow and stack-heavy synthetic cases, with speedups from $1.21\times$ (loop) up to $1.54\times$ (20).

These microbenchmarks clarify the mechanism behind these gains. The straight-line benchmarks show the largest improvements, with speedups growing as the sequence length increases. This is exactly where copy-and-patch should perform best: the hot path consists largely of repeated transitions among threaded functions, so replacing indirect threaded transfers with direct control flow yields a larger fraction of savings. The loop benchmark also improves significantly, although less dramatically than the straight-line cases. In that benchmark, the system still benefits from reduced dispatch overhead, but more of the total runtime is spent in actual loop work such as comparisons, arithmetic, and control-flow decisions, so the relative benefit is smaller.

One microbenchmark moves in the opposite direction. The arithmetic benchmark is slightly slower under copy-and-patch than under threaded execution. This result is useful because it shows that the benefit is not universal. When execution is dominated by very short primitive fast paths, dispatch

may already be a small fraction of total cost. In that setting, the additional code layout or control-flow structure introduced by template composition can offset or exceed the savings from reduced dispatch. The result supports a selective of the technique: copy-and-patch is most effective on dispatch-dominated paths, and less effective, or even counterproductive, on paths where primitive execution already dominates.

7.3. Evaluation

The microbenchmarks approximate favorable cases where internal threaded dispatch is a large share of the cost, and they show that copy-and-patch can deliver noticeably larger benefits there. Fibonacci, by contrast, represents a more conservative end-to-end workload in which dynamic sends and runtime bookkeeping remain prominent. The smaller but stable improvement on Fibonacci is therefore not at odds with the larger microbenchmark gains; it is the expected result once dispatch reduction is placed within a fuller Smalltalk execution path.

The machine context also matters. These measurements were taken on an Apple M2 system, whose branch prediction and front-end performance already reduce some of the penalty of indirect threaded execution. On such a machine, a consistent five percent improvement in a send-heavy workload is still meaningful, but it should not be interpreted as an upper bound on what the technique could achieve on other hardware. This interpretation is consistent with prior work showing that indirect branch behavior is a major performance factor in efficient interpreters [2, 3, 10].

Overall, the evaluation supports a pragmatic conclusion. Copy-and-patch is effective as a middle tier: it consistently improves performance when interpreter dispatch is a recurring component of execution, and it can produce substantially larger gains on dispatch-heavy microbenchmarks. However, its impact remains bounded in workloads where dynamic sends costs dominate.

8. Limitations

Dispatch Scope. The current design has a deliberately limited scope. Copy-and-patch reduces indirect transfers among many threaded functions, including stack operations, branches, inline primitives, and return paths, but it does not eliminate all dispatch overhead. In particular, dynamic Smalltalk message sends remain indirect by design. As a result, end-to-end gains on send-heavy workloads such as Fibonacci are necessarily limited, and the observed improvements should be interpreted as partial reduction of interpreter dispatch rather than complete removal of runtime dispatch costs.

Workload Dependent. The effectiveness of the technique also depends on workload structure. It is most beneficial when execution is dominated by frequent transitions among threaded functions, as reflected in the larger gains on dispatch-heavy microbenchmarks. By contrast, workloads dominated by dynamic sends or short primitive fast paths may see smaller benefits, and in some cases no benefit at all. The arithmetic microbenchmark illustrates this directly, suggesting that copy-and-patch is best understood as a selective rather than a universal one.

Portability Constraints The prototype is further constrained by implementation specificity. The current backend is specialized to AArch64 and depends on architecture-specific instruction decoding, relocation, branch encoding, and platform-specific executable-memory controls.

Evaluation Breadth. Finally, both coverage and evaluation breadth remain limited. The current implementation handles a practical subset of Zag threaded code, but not the full execution space of the VM. Likewise, the evaluation combines one send-heavy end-to-end workload with a set of controlled microbenchmarks, which is enough to demonstrate feasibility and explain the mechanism, but not enough to support broad claims across the full range of Smalltalk execution. The present system should therefore be viewed as a viable proof of concept for a middle-tier JIT, rather than a complete

production-quality architecture. Recent Smalltalk-specific work on inlined code generation suggests a natural next step for reducing the additional send-heavy costs [11].

9. Conclusion

This work presents a copy-and-patch JIT tier for the Zag Smalltalk VM that targets the gap between pure threaded execution and full optimizing JIT compilation. Instead of introducing a new backend pipeline, the approach reuses existing threaded functions as native-code templates, extracts patchable regions through control-flow-guided scanning, relocates PC-relative instructions, and patches control-flow edges into method-specific execution paths. The resulting design preserves threaded semantics while reducing repeated indirect dispatch.

The implementation demonstrates that this middle tier can be integrated with modest compiler complexity. In the evaluated prototype scope, copy-and-patch consistently improves Fibonacci runtime relative to the threaded baseline, with roughly stable percentage gains and increasing absolute savings as workload size grows. Microbenchmarks further clarify where the mechanism is most effective: dispatch-heavy paths see larger improvements, while very short primitive-dominated paths can show limited or negative benefit.

These outcomes support a practical conclusion. Copy-and-patch is a viable and useful acceleration tier for threaded dynamic-language runtimes, but its impact is currently bounded by what it does not optimize, including dynamic message-send resolution. The current system should therefore be viewed as a feasible middle tier rather than a replacement for full optimizing compilation.

Future Work. Future work should broaden opcode coverage, generalize template extraction and patch-site management, extend relocation/patching support beyond AArch64, and evaluate across a wider benchmark suite representative of full Smalltalk workloads. These steps would help establish when copy-and-patch is sufficient on its own and when it should serve as a staging tier toward more aggressive optimization.

References

- [1] M. A. Ertl, D. Gregg, A. Krall, B. Paysan, Vmgen—a generator of efficient virtual machine interpreters, *Software: Practice and Experience* 32 (2002) 265–294. URL: <https://doi.org/10.1002/spe.434>. doi:10.1002/spe.434.
- [2] M. A. Ertl, D. Gregg, The structure and performance of efficient interpreters, *Journal of Instruction-Level Parallelism* 5 (2003) 1–25. URL: <http://www.jilp.org/vol5/v5paper12.pdf>.
- [3] K. Casey, M. A. Ertl, D. Gregg, Optimizing indirect branch prediction accuracy in virtual machine interpreters, *ACM Transactions on Programming Languages and Systems* 29 (2007) 37:1–37:38. URL: <https://mural.maynoothuniversity.ie/id/eprint/10188/>. doi:10.1145/1286821.1286828.
- [4] H. Xu, F. Kjolstad, Copy-and-patch compilation: A fast compilation algorithm for high-level languages and bytecode, *Proceedings of the ACM on Programming Languages* 5 (2021) 136:1–136:30. URL: <https://openreview.net/forum?id=dZ6c3CKBHW>. doi:10.1145/3485513.
- [5] J. Baig, D. Mason, Smalltalk jit compilation: Llmv experimentation, in: *International Workshop on Smalltalk Technologies 2024 (IWST 2024)*, 2024. URL: <https://ceur-ws.org/Vol-3893/Paper01.pdf>.
- [6] L. P. Deutsch, A. M. Schiffman, Efficient implementation of the smalltalk-80 system, in: *Conference Record of the Eleventh Annual ACM Symposium on Principles of Programming Languages*, 1984, pp. 297–302. URL: <https://doi.org/10.1145/800017.800542>. doi:10.1145/800017.800542.
- [7] E. Miranda, The cog smalltalk virtual machine—writing a jit in a high-level dynamic language, 2011. URL: <https://doi.org/10.5281/zenodo.3700608>. doi:10.5281/zenodo.3700608, workshop paper.
- [8] D. Mason, Threaded execution as a dual to native code, in: *Companion Proceedings of the 7th*

- International Conference on the Art, Science, and Engineering of Programming, 2023, pp. 7–11. URL: <https://dblp.org/rec/conf/programming/Mason23>. doi:10.1145/3594671.3594673.
- [9] Arm Limited, Arm Architecture Reference Manual for A-profile Architecture, Arm Limited, 2023. URL: <https://developer.arm.com/documentation/ddi0487/latest>, arm DDI 0487.
- [10] D. Mason, Revisiting dynamic dispatch for modern architectures, in: Proceedings of the 15th ACM SIGPLAN International Workshop on Virtual Machines and Intermediate Languages, 2023, pp. 11–17. URL: <https://researchr.org/publication/vmil-2023>. doi:10.1145/3623507.3623551.
- [11] D. Franklin, D. Mason, Inlined code generation for smalltalk, in: International Workshop on Smalltalk Technologies 2024 (IWST 2024), 2024. URL: <https://ceur-ws.org/Vol-3893/Paper06.pdf>.